Clase D01 · De Análisis a Diseño + Principios

Bloque 2 · Diseño · Clase 1 de 9

Modelación y Diseño de Sistemas · 2026 · Bloque Diseño

Prof. Ing. Fernando Bono · ESCMB UNC · Pregrado AUSI

Agenda

- Mapa: dónde estamos (cerramos Análisis, abrimos Diseño)
- Análisis vs Diseño: ¿qué cambia?
- Recap: las 5 vistas del Parcial 1 como input
- ¿Qué es el diseño? ¿Por qué importa?
- Los 4 principios fundamentales
- Abstracción
- Modularidad
- Cohesión (alta vs baja)
- Acoplamiento (bajo vs alto)
- La regla de oro
- Transición Análisis → Diseño paso a paso
- Responsabilidades: doing vs knowing (Larman)
- Colaboraciones entre objetos
- Caso conductor ChangoMás: aplicando los principios al UC "Procesar venta"
- Errores comunes al diseñar OO
- Cierre + qué viene en D02

1 · ¿Dónde estamos?

El mapa de la materia

Bloque	Estado
B0 · Introducción + Importancia de modelar	✓ cerrado
B1 · Análisis (Contexto, UC, Modelo de Dominio, SSD, Estados)	✓ cerrado
Parcial 1 · cierre del bloque de Análisis	✓ rendido
B2 · Diseño ← ESTAMOS ACÁ	en curso
B3 · Arquitectura	sigue
B4 · Testing + CI/CD	sigue
Parcial 2 · cierre integral	sigue

Hasta acá analizamos QUÉ debe hacer el sistema. A partir de ahora diseñamos CÓMO hacerlo.

2 · Análisis vs Diseño: la diferencia clave

	Análisis	Diseño
Pregunta	¿QUÉ debe hacer el sistema?	¿CÓMO lo hace internamente?
Foco	El problema, el dominio	La solución, el software
Vista	Caja negra	Caja blanca
Artefactos	Contexto, UC, Modelo de Dominio, SSD, Estados	Diagrama de Clases de Diseño (DCD), Diagrama de Secuencia de Diseño (DSeD)
Lenguaje	Del negocio (Cliente, Caja, Pedido)	Del software (Controller, Service, Repository)
Métodos en clases	NO	SÍ

El diseño no descarta lo anterior: lo transforma y extiende.

3 · Recap Parcial 1: lo que tenemos como entrada

Las 5 vistas que cerramos en el bloque de Análisis son el input del diseño:

- Diagrama de Contexto: el sistema como caja negra rodeado de actores (Cliente, Cajero, Pasarela de Pago, Impresora).
- Diagrama de Casos de Uso: actores y funcionalidades visibles.
- Especificaciones fully-dressed (Cockburn): flujos principal y alternativos por UC.
- Modelo de Dominio: conceptos del negocio con atributos y asociaciones (sin métodos).
- SSD: eventos actor ↔ sistema, una sola línea de vida del lado del sistema.
- Diagrama de Estados: ciclo de vida de entidades clave (Venta, Pedido).

Estos artefactos NO se tiran. Son la materia prima del bloque de Diseño.

4 · ¿Qué es el diseño de software?

- Actividad creativa de proponer una solución al problema definido en el análisis.
- Decide la estructura interna del sistema: qué clases existen y cómo cooperan.
- Trabaja a nivel de software, no de dominio: aparecen Controller, Service, Repository, DAO.
- Genera modelos que sirven de puente entre el análisis y el código.
- Es iterativo: las primeras versiones se refinan a medida que se entiende mejor.

¿Por qué importa?

- Sin diseño explícito, el código termina desorganizado, frágil y caro de mantener.
- El diseño guía la asignación de responsabilidades: cada clase tiene un propósito claro.
- Permite anticipar problemas (cuellos de botella, dependencias, decisiones costosas).
- Facilita la comunicación entre desarrolladores.
- Reduce el costo de cambios futuros.

Diseñar mal cuesta más caro que diseñar bien. La clase de hoy es para ahorrar plata futura.

5 · Los 4 principios fundamentales

Los dos primeros son técnicas, los dos últimos son métricas.

Principio	Pregunta clave
Abstracción	¿Qué muestro y qué oculto?
Modularidad	¿Cómo divido el sistema en partes manejables?
Cohesión	¿Qué tan relacionadas están las responsabilidades dentro de un módulo?
Acoplamiento	¿Qué tan dependientes son los módulos entre sí?

Principio 1 · Abstracción

- Mostrar solo lo esencial, ocultar el resto.
- Una clase MedioPago expone cobrar(monto) pero no expone si por dentro habla con Visa, Mastercard o Mercado Pago.
- Permite usar un módulo sin conocer su implementación interna.
- Es la base de la encapsulación y de las interfaces.
- Niveles: abstracción de datos (clases), de procedimientos (funciones), de control (eventos).

Ejemplo ChangoMás:

El POS habla con MedioPago, no con cada implementación.

```
interface MedioPago:
    cobrar(monto: Money): Comprobante

class PagoEfectivo implements MedioPago
class PagoTarjeta implements MedioPago
class PagoQR     implements MedioPago
```

Principio 2 · Modularidad

- Dividir el sistema en módulos (clases, paquetes, capas) que se diseñan por separado.
- Cada módulo cumple una responsabilidad bien definida.
- Beneficios: trabajo en paralelo del equipo, testing aislado, comprensión más simple.
- Demasiada modularidad → fragmentación + complejidad de coordinación.
- Poca modularidad → "God Classes" (una clase que hace todo).

Ejemplo ChangoMás:

Cada paquete tiene su propio equipo, sus propios tests, su propio deploy posible.

```
ventas/          ← lógica de la venta
inventario/      ← stock y reposición
clientes/        ← padrón y fidelización
pagos/           ← integraciones con pasarelas
infra/           ← BD, mensajería, dispositivos
```

Principio 3 · Cohesión

Qué tan relacionadas están las responsabilidades de UN módulo.

- Cohesión ALTA (deseable): el módulo hace UNA cosa, bien.
    - Ej.: CalculadoraImpuestos que solo calcula impuestos.
- Cohesión BAJA (a evitar): el módulo hace muchas cosas distintas.
    - Ej.: Utils con parseFecha(), enviarEmail(), calcularIVA(), formatearMoneda().

Test de cohesión: ¿cómo se llama esta clase? Si el nombre necesita "And" (UsuarioYPedidoManager), tiene baja cohesión.

Cohesión · ejemplo ChangoMás

BAJA cohesión (a evitar):

Si cambia el formato del email, hay que tocar la misma clase que procesa ventas.

ALTA cohesión (deseable):

Cada clase hace UNA cosa. Cambios localizados.

```
class GestorCaja:
    procesarVenta()
    enviarComprobantePorEmail()
    generarReporteDiario()
    conectarBaseDeDatos()
    validarDNI()
    consultarStock()
```

Principio 4 · Acoplamiento

Qué tan dependientes son DOS o más módulos entre sí.

- Acoplamiento BAJO (deseable): los módulos saben lo mínimo uno del otro.
- Se comunican por interfaces claras y estables.
- Acoplamiento ALTO (a evitar): un cambio en uno obliga a cambiar al otro.
- Genera efecto dominó: tocar una clase rompe varias.

Tipos (de menor a mayor): por datos < por estructura < por control < común < de contenido.

Acoplamiento · ejemplo ChangoMás

ALTO acoplamiento:

Cambiar de SMTP a SendGrid ⇒ modificar GestorVenta (que no tiene nada que ver con email).

BAJO acoplamiento:

Notificador puede usar SMTP, SendGrid, WhatsApp o paloma mensajera. GestorVenta no se entera.

```
class GestorVenta:
    procesarVenta(venta):
        # GestorVenta CONOCE los detalles del SMTP
        smtp = SMTP("mail.changomas.com.ar", 587)
        smtp.login(USER, PASS)
        smtp.sendmail(cliente.email, "Comprobante", venta.html)
```

6 · La regla de oro

Estos dos principios van juntos. Si los aplicás bien, el resto sigue casi solo.

Beneficios concretos:

- Código más fácil de testear (módulos aislables, se pueden mockear las dependencias).
- Más fácil de mantener (cambios localizados).
- Más fácil de reusar (módulos independientes).
- Más fácil de entender (cada clase cuenta su propia historia).

ALTA cohesión + BAJO acoplamiento

7 · Transición Análisis → Diseño paso a paso

Artefacto de Análisis	Se transforma en	Cambio principal
Modelo de Dominio	Diagrama de Clases de Diseño (DCD)	Las clases conceptuales suman métodos, visibilidad, tipos. Aparecen clases nuevas: controladores, servicios, repositorios, fachadas.
SSD	Diagrama de Secuencia de Diseño (DSeD)	La "caja negra" del sistema se abre. Aparecen las colaboraciones internas. Cada operación se distribuye entre varios objetos.
Casos de Uso	Guían el diseño, no se transforman	Cada UC se traduce en una colaboración entre clases de diseño.
Diagrama de Estados	Lifecycle de la entidad de diseño	Se traduce a una máquina de estados implementable (enum + transiciones + guardas).

Pregunta del diseñador: "Para cumplir esta operación del sistema, ¿qué objetos necesito y qué hace cada uno?"

8 · Responsabilidades: doing vs knowing

Una responsabilidad es aquello que un objeto sabe hacer o sabe contestar.

- NO es un método. Es una abstracción de mayor nivel: un compromiso del objeto.
- Una responsabilidad puede traducirse en uno o varios métodos.

Dos tipos (Larman)

- DOING (hacer): acciones que realiza el objeto.
  - Crear otro objeto, calcular algo, coordinar actividades, delegar trabajo.
  - Ej.: Venta.calcularTotal(), Cajero.procesarItem(producto)
- KNOWING (conocer): información que mantiene el objeto.
  - Atributos privados, datos sobre objetos relacionados, datos derivados.
  - Ej.: Cliente.dni, Venta.fecha, Producto.stockDisponible

> *"Diseñar es asignar responsabilidades a clases." — Craig Larman, UML y Patrones (cap. 17)*

El reto: asignar bien

- Una operación del sistema (ej.: "procesar venta") no la hace un solo objeto: la hacen varios colaborando.
- El diseñador decide: ¿qué pasos hace cada objeto? ¿quién coordina? ¿quién informa?
- Mala asignación → diseño rígido, frágil, difícil de cambiar (alto acoplamiento, baja cohesión).
- Buena asignación → diseño limpio, flexible, fácil de extender.
- Hay patrones (GRASP) que dan reglas para asignar bien — los vemos en D02.

Esta es la habilidad central del diseñador OO: saber a quién preguntarle. Cuesta práctica adquirirla.

9 · Colaboraciones entre objetos

- Una colaboración es la interacción entre dos o más objetos para cumplir un objetivo.
- Un objeto le pide a otro: "hacé esto" o "decime esto otro" (mensaje).
- El receptor cumple su responsabilidad y devuelve el resultado si corresponde.
- Las colaboraciones forman cadenas: A pide a B, B pide a C, C devuelve a B, B devuelve a A.
- Cada operación del sistema (del SSD) se traduce en una colaboración entre varios objetos.

Mensajes y delegación

- Mensaje: la forma de pedir algo a otro objeto. En código, una invocación de método.
- Delegación: pasar el trabajo a otro objeto que está mejor posicionado para hacerlo.

Ejemplo ChangoMás:

GestorVenta.procesarVenta() le pide a Cliente.puedeComprarFiado().

El gestor no chequea internamente al cliente: delega esa responsabilidad al propio Cliente.

Si cambia la regla "puede comprar fiado", solo se toca la clase Cliente.

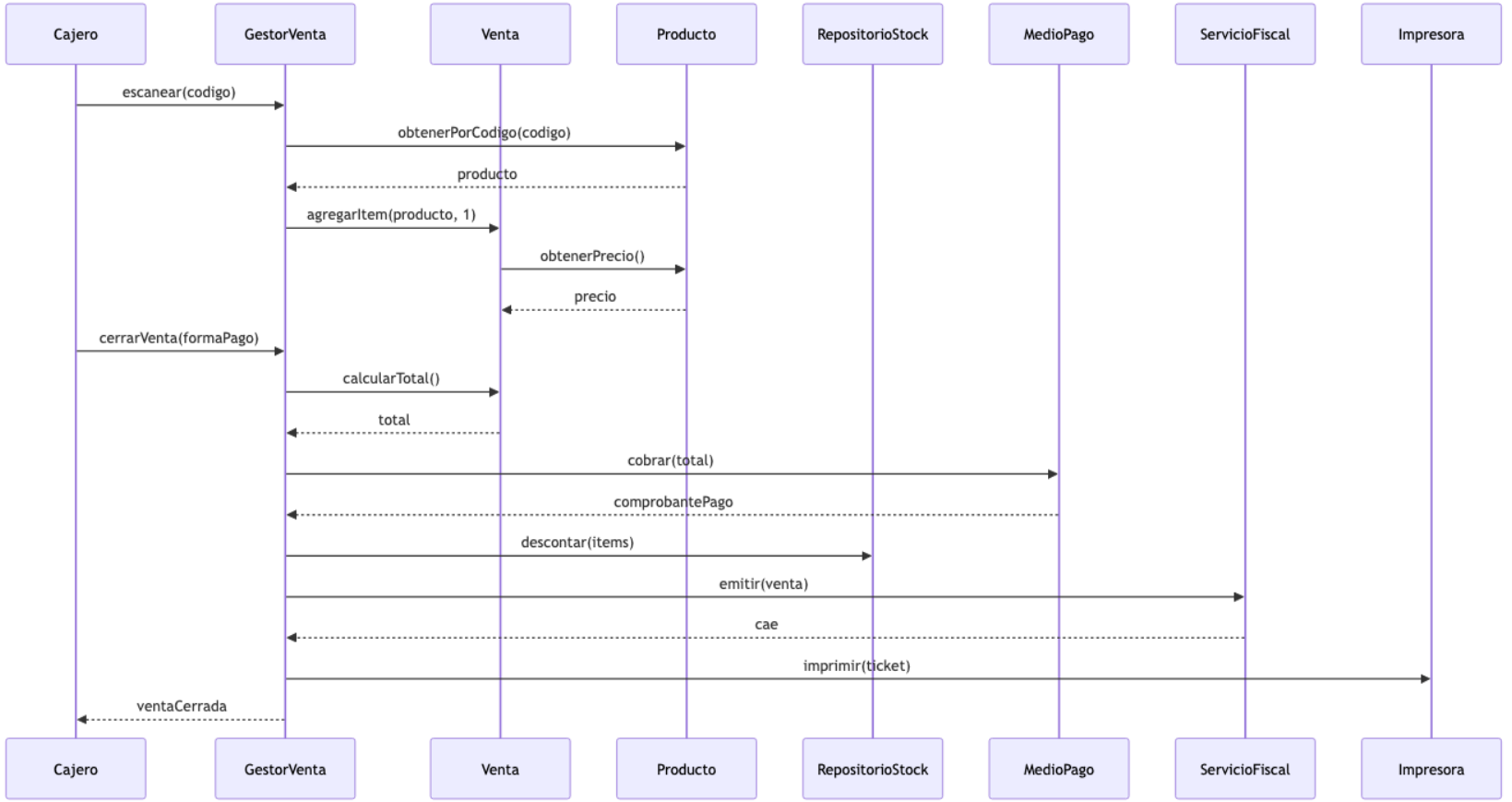
Paso 1 · Identificar responsabilidades

Para el UC "Procesar venta en caja" necesitamos:

- Recibir cada producto escaneado por el cajero.
- Verificar que el producto exista y tenga stock.
- Sumar el ítem al ticket.
- Calcular el subtotal, los impuestos y el total.
- Procesar el pago (efectivo, tarjeta, QR).
- Descontar el stock del depósito.
- Emitir el comprobante fiscal.
- Imprimir el ticket.
- Cerrar la venta.

Pregunta clave: ¿quién hace cada cosa?

Paso 3 · Diagrama de Secuencia de Diseño (vista resumen)



Paso 4 · Diagrama de Clases de Diseño (vista textual)

```
GestorVenta {
    + iniciar(cajero): Venta
    + escanear(codigo): void
    + cerrarVenta(formaPago): Comprobante
}

Venta {
    - id: int
    - fecha: Date
    - estado: EstadoVenta
    - items: List<Item>
    + agregarItem(producto, cantidad): void
    + calcularTotal(): Money
}

Producto {
    - codigo: String
    - nombre: String
    - precio: Money
    - iva: Decimal
    + obtenerPrecio(): Money
    + tieneStock(cantidad): bool
}

RepositorioStock {
    + consultar(producto): int
    + descontar(items: List<Item>): void
}

interface MedioPago {
    + cobrar(monto: Money): Comprobante
}

class PagoEfectivo  implements MedioPago
class PagoTarjeta   implements MedioPago
class PagoQR        implements MedioPago

ServicioFiscal {
    + emitir(venta: Venta): CAE
}

Impresora {
    + imprimir(ticket: Ticket): void
}
```

Hay alta cohesión (cada clase hace una cosa) y bajo acoplamiento (las dependencias pasan por interfaces).

11 · Errores comunes al diseñar OO

- Hacer God Classes que concentran toda la lógica (baja cohesión).
- Confundir Modelo de Dominio con Diagrama de Clases (poner métodos en el MoD).
- No delegar: pedir datos a otro objeto y decidir afuera (en vez de pedirle que decida).
- Acoplar a clases concretas en vez de a interfaces (frágil ante cambios).
- Nombrar las clases por rol genérico (Manager, Helper, Util) — no dicen nada.
- Saltarse el modelo de diseño y empezar a codear — terminás con espagueti.

Buen diseño = clases con nombres claros, responsabilidades claras y colaboraciones explícitas.

12 · Cierre + qué viene

Lo que vimos hoy

- El diseño es la primera transformación del análisis a la solución concreta.
- Los 4 principios se interconectan: abstracción + modularidad llevan a alta cohesión y bajo acoplamiento.
- Cohesión alta = la clase hace UNA cosa, bien.
- Acoplamiento bajo = los módulos se comunican por interfaces.
- La transición A→D no descarta lo anterior: lo refina con clases de software, métodos y delegaciones.
- Diseñar = asignar responsabilidades (doing/knowing) y orquestar colaboraciones.

El bloque de Diseño en MyD 2026

Bibliografía

- Larman, C. — UML y Patrones (2da ed.), caps. 16 y 17.
- Sommerville, I. — Ingeniería de Software (9na ed.), cap. 7.
- Pressman, R. — Ingeniería de Software, un enfoque práctico (7ma ed.), cap. 8.

12 · Cierre + qué viene

Clase	Tema
D01 ← hoy	Análisis → Diseño + Principios
D02	Patrones GRASP (5 básicos + 4 avanzados)
D03	Patrones GoF (Singleton, Observer, Strategy, Factory, Decorator, Adapter)
D04	Arquitectura: conceptos + atributos de calidad + ADRs
D05	Estilos clásicos: Capas + Cliente-Servidor
D06	Estilo MVC
D07	Microservicios
D08	Event-Driven
D09	Arquitecturas modernas: Clean + Hexagonal + Onion

Próxima clase (D02): Patrones GRASP — las recetas para asignar responsabilidades.